FinBuddy

Smart Expense Splitting & Settlement System

Hackathon	BREACH PDEU 2026
Team Name	DhuranDhars
Team Leader	Harshil Mistry
Team Members	*Dharmil Gadhiya* *Meet Borkhatariya* *Vraj Patel* *Jay Rathod*
Date	March 14, 2026
Institute	Charotar University of Science and Technology
Repository	harshil-mistry/FinBuddy-breach-pdeu

Version 1.0.0 — Flutter (Dart) — Firebase — Node.js

Contents

1 Abstract / Executive Summary **3**

2 Problem Statement **3**
2.1 What Financial Issue Exists . . . 3
2.2 Who Is Affected . . . 3
2.3 Why Current Systems Fail . . . 4

3 Objectives **4**

4 Proposed Solution **4**
4.1 Working Mechanism . . . 4
4.1.1 Core Algorithm: Debt Simplification (Graph-Based Greedy Matching) . . . 4
4.1.2 Smart Feature: AI-Powered Expense Logging . . . 5
4.2 Product Design . . . 5
4.3 User Journey . . . 6

5 System Architecture **6**
5.1 High-Level Data Flow . . . 6
5.2 System Components . . . 7
5.3 Model Pipeline . . . 7
5.3.1 Expense Logging Pipeline . . . 7
5.3.2 Settlement Optimisation Pipeline . . . 8

6 Technology Stack **8**
6.1 Frontend . . . 8
6.2 Backend . . . 8
6.3 Database & Authentication . . . 8
6.4 AI & Processing Services . . . 9
6.5 Payment Integration . . . 9
6.6 Deployment & Infrastructure . . . 9

7 Implementation **9**
7.1 Prototype Development Process . . . 9
7.2 Important Modules . . . 9
7.2.1 Authentication Module . . . 9
7.2.2 Expense Management Module . . . 10
7.2.3 Group Management Module . . . 10
7.2.4 Settlement Optimisation Module . . . 10
7.2.5 Smart Logging Module . . . 10
7.2.6 UI / UX Screens . . . 10
7.2.7 Firebase Integration Module . . . 10
7.3 Algorithms Used . . . 10
7.3.1 Debt Simplification Algorithm . . . 10
7.3.2 Expense Split Calculation . . . 11
7.3.3 VLM Receipt Scanning (llama-3.2-vl) . . . 11
7.3.4 QR Code Data Encoding . . . 11

8 Results / Demo **11**
8.1 Screenshots 12
8.2 Prototype Output 13
8.3 Sample Transactions 13

9 Business Model **13**

10 Challenges Faced **14**

11 Future Scope **14**

12 Conclusion **15**

13 References **15**

1. Abstract / Executive Summary

Executive Summary

Group expense management is a persistent pain point for students, travellers, and communities worldwide. Manual tracking via spreadsheets or chat messages is error-prone and opaque. **FinBuddy** is a cross-platform mobile application that transforms this experience through intelligent expense splitting, AI-powered logging, and mathematically optimised settlement calculations.

Problem. Splitting shared costs—travel, meals, hostel fees—among multiple parties is tedious and often leads to inaccuracies, delayed settlements, and broken trust. No widely-used lightweight solution integrates multi-party balance tracking with automated settlement optimisation and smart input methods.

Solution. FinBuddy allows users to create groups, log expenses via voice or a Vision-Language Model (VLM) receipt scan, and automatically derives the minimum number of peer-to-peer transactions needed to fully clear all debts using a greedy directed-graph algorithm. Direct UPI deep-links eliminate friction at the payment step.

Impact. FinBuddy reduces financial confusion in shared-expense scenarios, promotes transparency through an immutable Firestore ledger, minimises the number of settlement transactions, and accelerates payment closure through native UPI integration.

2. Problem Statement

2.1. What Financial Issue Exists

When groups of people share expenses, calculating individual debts and determining optimal settlement paths becomes combinatorially complex. The number of required peer-to-peer transactions grows with group size, yet informal tools provide no mechanism to simplify or optimise them.

2.2. Who Is Affected

- College students and hostel communities managing shared household costs
- Friend groups planning domestic or international trips
- Event organisers balancing group catering, transport, and venue expenses
- Roommates tracking recurring utility and grocery bills
- Any informal group engaging in shared financial activity

2.3. Why Current Systems Fail

Table 1: Limitations of Existing Solutions

LightBlue **Method**	**Limitation**
Manual Spreadsheets	Time-consuming, error-prone, no real-time access or collaboration
Generic Payment Apps	Designed for single-transaction transfers; no multi-party ledger
Messaging Apps	Unstructured; no audit trail, no algorithmic settlement
Basic Expense Apps	Lack settlement-optimisation; limited input methods

3. Objectives

FinBuddy is built to achieve the following objectives:

1. **Efficient Group Expense Tracking** — Create and manage group expenses with full member visibility and an immutable transaction ledger.
2. **Intelligent Expense Splitting** — Support equal, percentage-based, and custom split methods with automatic share calculation.
3. **Optimised Settlement Calculations** — Apply a directed-graph greedy algorithm to derive the minimum set of transactions required to settle all debts.
4. **Smart Expense Logging** — Enable voice-based entry (speech-to-text + LLM parsing) and image-based receipt scanning via a Vision-Language Model (VLM).
5. **Digital Payment Integration** — Trigger UPI payments with pre-filled recipients and amounts through native deep-linking.
6. **Security and Cross-Platform Availability** — Firebase Auth for secure authentication; Flutter for native-quality iOS and Android apps from a single codebase.

4. Proposed Solution

4.1. Working Mechanism

4.1.1. Core Algorithm: Debt Simplification (Graph-Based Greedy Matching)

The system models group debts as a **directed weighted graph**:

- **Nodes** represent individual users in a group.
- **Edges** represent a debt from one user to another, weighted by the owed amount.

The algorithm proceeds in four steps:

1. Compute **net balance** for every user: $\text{netBal}(u) = \text{TotalIn}(u) - \text{TotalOut}(u)$.
2. Partition users into *debtors* ($\text{netBal} < 0$) and *creditors* ($\text{netBal} > 0$).
3. Greedily match the **largest debtor** D with the **largest creditor** C; settle $\min(|D|, |C|)$.
4. Repeat until all balances reach zero.

Result: The minimum number of transactions required to settle all inter-group debts.

4.1.2. Smart Feature: AI-Powered Expense Logging

Voice-to-Expense Workflow

1. User records an audio clip describing the expense.
2. **Deepgram API** transcribes audio to text.
3. **Google Gemini API** parses the transcript and returns structured JSON: `{"amount": 800, "payer": "Aman", "split": "equal"}`
4. The Express backend validates the JSON and returns it to Flutter.
5. User confirms before writing to Firestore.

VLM Receipt Scanning Workflow

1. User captures a photo of the receipt within the app.
2. The image is sent directly to **llama-3.2-vl VLM** via the Model-compatible API.
3. The VLM extracts line items, prices, taxes, and totals as structured JSON.
4. User reviews and assigns items to group members.
5. System calculates individual portions and writes to Firestore.

This approach is superior to classical OCR: the VLM understands layout, handles distortion, and can reason about partial or crumpled receipts without dedicated preprocessing pipelines.

4.2. Product Design

FinBuddy consists of three architectural layers:

1. **Client Layer (Flutter Mobile App)** — Local state via Provider; Firebase SDK; audio/image capture; QR code generation & scanning; debt graph visualisation; UPI deep-link integration.
2. **Backend API Layer (Node.js + Express)** — Firebase token verification middleware; graph algorithm execution; AI service orchestration (Deepgram, Gemini, GPT VLM); secure `.env` key storage.
3. **Data Layer (Firebase)** — Firestore NoSQL database for users, groups, expenses, and transactions; Firebase Auth for token management; immutable ledger for audit trails.

4.3. User Journey

Step 1: Onboarding & Group Creation — Sign up via email or Google; dashboard shows Personal and Shared expense tabs; create a group with a name.

Step 2: Invite Members — App generates a unique group QR code; friends scan it with the in-app scanner to join instantly.

Step 3: Log an Expense — Choose Manual, Voice, or Receipt Scan (VLM); select split method; confirm shares.

Step 4: View Ledger — Dashboard shows all transactions; filter by date, category, or member; personal analytics panel.

Step 5: Settle Debts — Navigate to *Settlement* tab; app runs the graph optimisation; one-tap *Pay Now* opens UPI app pre-filled with recipient and amount.

Step 6: Confirm Settlement — Mark payment complete in FinBuddy; ledger updates and remaining debts are recalculated automatically.

5. System Architecture

5.1. High-Level Data Flow

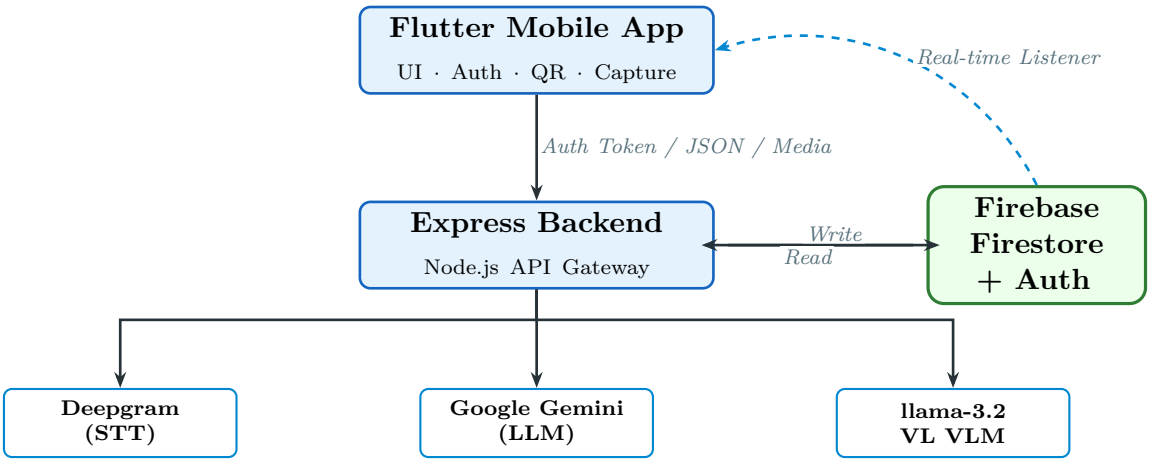


Figure 1: High-Level Data Flow between FinBuddy Components

5.2. System Components

Table 2: Technology Mapping per System Component

LightBlue **Component**	**Technology**	**Purpose**
Mobile Frontend	Flutter (Dart)	Cross-platform iOS & Android app
Backend Server	Node.js + Express	API gateway, business logic, AI orchestration
Database	Firebase Firestore	Transaction ledger, user & group data
Authentication	Firebase Auth	Sign-up, login, token management
Speech Recognition	Deepgram API	Audio transcript generation
LLM Parsing	Google Gemini API	Extract structured data from transcript
Receipt Scanning	llama-3.2-vl VLM	End-to-end image-to-JSON receipt extraction
Payment Integration	UPI Deep-linking	Native payment app triggering

5.3. Model Pipeline

5.3.1. Expense Logging Pipeline

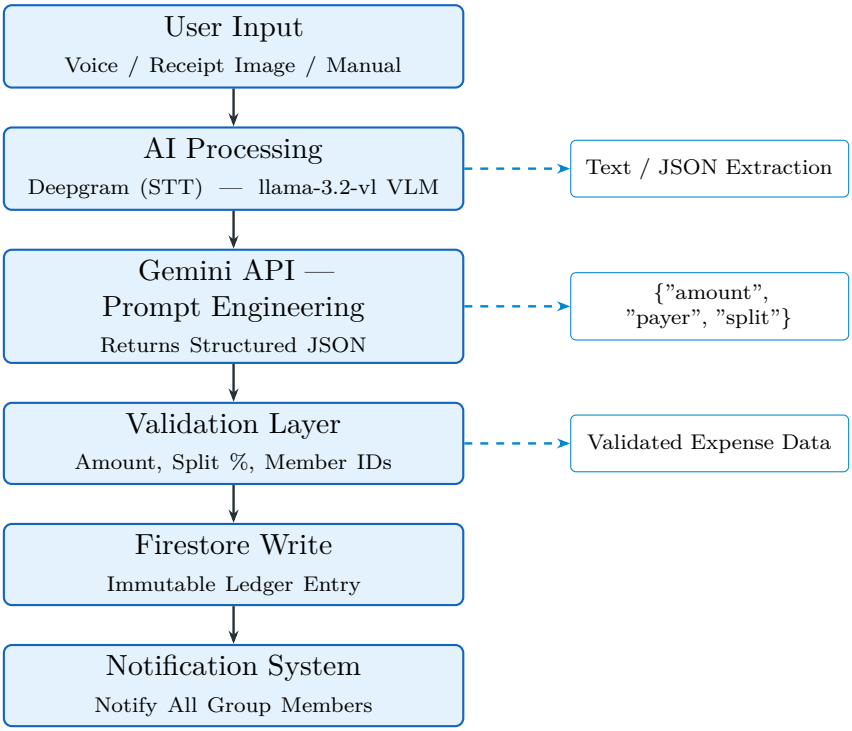


Figure 2: AI-Powered Expense Logging Pipeline

5.3.2. Settlement Optimisation Pipeline

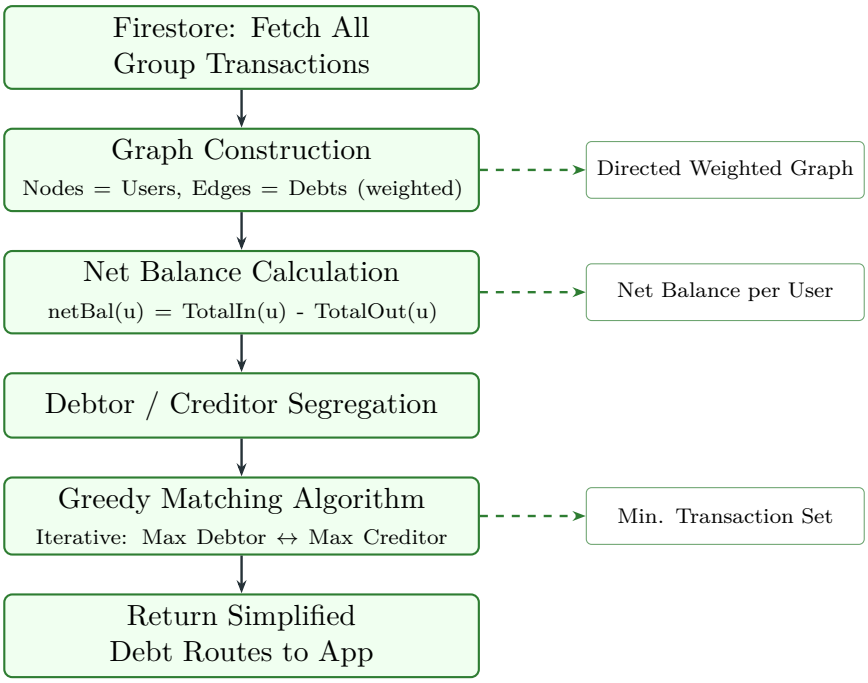


Figure 3: Settlement Optimisation Pipeline

6. Technology Stack

6.1. Frontend

- **Framework:** Flutter (Dart SDK `^3.8.1`)
- **State Management:** Provider v6.1.5
- **Key Libraries:** `firebase_core` · `firebase_auth` · `cloud_firestore` · `google_sign_in` · `qr_flutter` · `mobile_scanner` · `url_launcher` · `google_fonts` · `intl` · `uuid`

6.2. Backend

- **Runtime:** Node.js
- **Framework:** Express.js
- **Responsibilities:** API gateway, Firebase token verification middleware, graph algorithm execution, AI service orchestration, secure `.env` key storage

6.3. Database & Authentication

- **Database:** Firebase Firestore — document-based NoSQL with real-time listeners and offline persistence
- **Auth:** Firebase Auth — Email/Password, Google Sign-In, token validation

6.4. AI & Processing Services

Table 3: AI Services Integrated in FinBuddy

LightBlue **Service**	**Provider**	**Use Case**
Speech-to-Text	Deepgram API	Convert voice input to text transcript
Language Model	Google Gemini API	Parse text and extract structured expense JSON
Vision-Language	llama-3.2-vl VLM	Direct image-to-JSON receipt data extraction

6.5. Payment Integration

UPI Deep-linking (Android Intent / iOS Universal Link) — pre-fills recipient UPA, amount, and reference note; compliant with the NPCI UPI protocol.

6.6. Deployment & Infrastructure

Firebase hosting for Auth and Firestore; Node.js backend deployable on AWS Lambda; Flutter build system targeting iOS (CocoaPods) and Android (Gradle).

7. Implementation

7.1. Prototype Development Process

Phase 1: Foundation Setup — Flutter project initialisation with Material Design 3; Firebase project and authentication configuration; local dev environment.

Phase 2: Core Features — User authentication (Email + Google); group creation and QR code generation/scanning; manual expense entry forms; expense ledger display.

Phase 3: Smart Features — Deepgram STT integration; Google Gemini LLM parsing pipeline; llama-3.2-vl VLM integration for receipt scanning (replacing classical OCR).

Phase 4: Settlement Logic — Directed graph data structure; debt simplification algorithm; settlement UI.

Phase 5: Integration & Polish — UPI deep-linking; Firestore real-time listeners; notification system; input validation and error handling.

7.2. Important Modules

7.2.1. Authentication Module

`lib/services/auth_service.dart` — Handles registration, login (email + Google), token management, session persistence, and Firebase token verification forwarded to the Express backend.

7.2.2. Expense Management Module

`lib/services/expense_service.dart` — CRUD on expense records; split calculation (equal / percentage / custom); expense tagging (personal vs. shared).

7.2.3. Group Management Module

`lib/services/group_service.dart` — Group creation; QR code generation (`qr_flutter`); QR scanning (`mobile_scanner`); member add/remove; group data retrieval.

7.2.4. Settlement Optimisation Module

`lib/algorithms/debt_optimizer.dart` — Directed graph construction; net-balance calculation; debtor/creditor partitioning; greedy matching; returns minimum transaction set.

7.2.5. Smart Logging Module

`lib/services/smart_logger.dart` — Orchestrates Deepgram (voice), Gemini (parsing), and llama-3.2-vl VLM (receipt); returns validated structured expense JSON to UI.

7.2.6. UI / UX Screens

`lib/screens/` — `DashboardScreen`, `GroupsScreen`, `ExpenseEntryScreen`, `LedgerScreen`, `SettlementScreen`, `ProfileScreen`.

7.2.7. Firebase Integration Module

`lib/services/firestore_service.dart` — CRUD on Firestore; real-time `Stream` listeners; atomic batch writes; offline persistence.

7.3. Algorithms Used

7.3.1. Debt Simplification Algorithm

```
Algorithm: OptimizeSettlements(Transactions)
Input:  List of all transactions T in a group
Output: Minimum set of simplified transactions S

1.  Build directed graph G from T
2.  For each user U:
        netBalance[U] = sum(inflows) - sum(outflows)
3.  Debtors  := { U | netBalance[U] < 0 }
    Creditors := { U | netBalance[U] > 0 }
4.  S := []
    While Debtors != {} AND Creditors != {}:
        D := argmax { |netBalance[u]| : u in Debtors  }
        C := argmax {  netBalance[u]  : u in Creditors }
        amount := min( |netBalance[D]|, netBalance[C] )
        S.append( Transaction(D -> C, amount) )
        netBalance[D] += amount
        netBalance[C] -= amount
        Remove zero-balance users from their sets
5.  Return S

Time  Complexity: O(n log n)   -- priority-queue based
   greedy
```

```
Space Complexity: O(n)          -- balance array + graph
```

Listing 1: Greedy Debt Simplification — Pseudocode

7.3.2. Expense Split Calculation

Equal Split:

$$\text{share}_i = \frac{\text{totalAmount}}{|\text{members}|} \quad \forall\, i$$

Percentage Split:

$$\text{share}_i = \frac{p_i}{100} \times \text{totalAmount}, \qquad \sum_i p_i = 100$$

Custom Split:

$$\text{share}_i = c_i, \qquad \sum_i c_i = \text{totalAmount}$$

7.3.3. VLM Receipt Scanning (llama-3.2-vl)

The receipt image is passed directly to llama-3.2-vl with a structured prompt requesting a JSON array of line items. The model reasons over the image layout, handles distorted or partial receipts, and returns:

```
{"items": [{"name": "Pasta", "price": 280}, ...], "total": 1140}
```

This eliminates all classical OCR pre-processing (binarisation, deskew, segmentation) and achieves higher accuracy on real-world receipts.

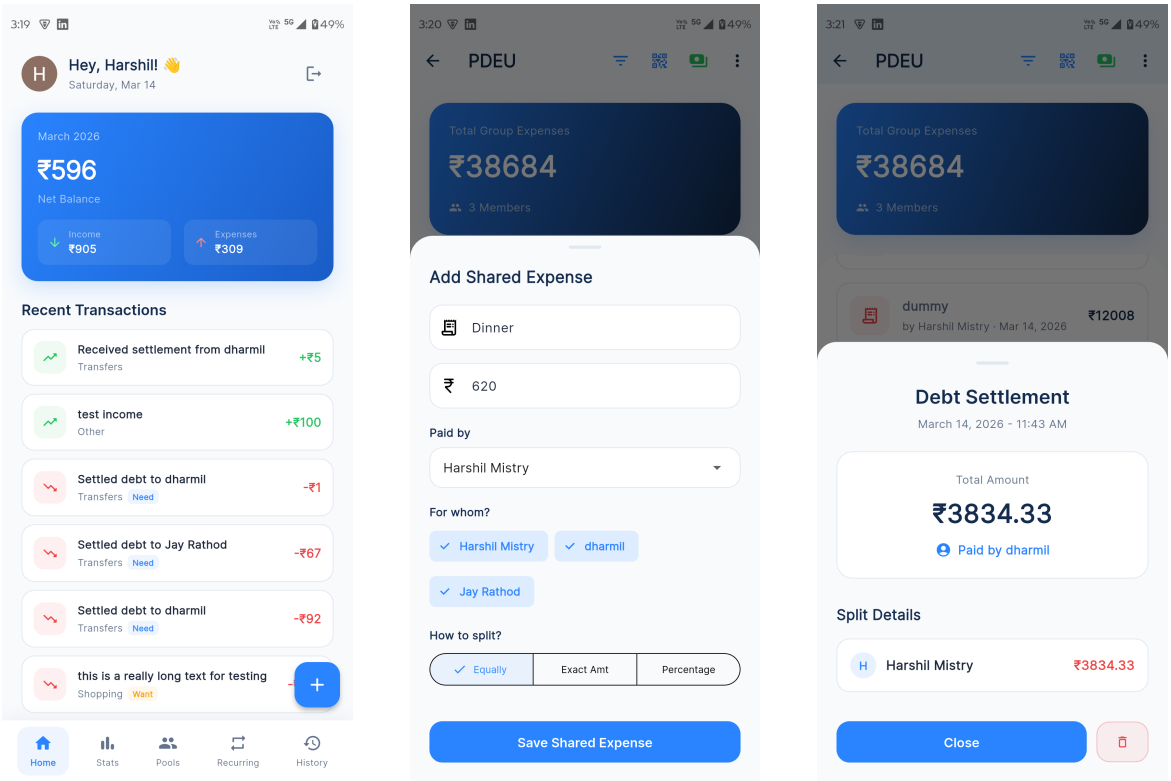
7.3.4. QR Code Data Encoding

Group invitation data is JSON-serialised, then encoded as a QR code:

```
{"groupId": "...", "groupName": "...", "creatorId": "...", "ts":
                    "ISO8601", "v": "1.0"}
```

8. Results / Demo

8.1. Screenshots



(a) Dashboard (b) Expense Entry (c) Settlement View

Figure 4: FinBuddy Application Screens

8.2. Prototype Output

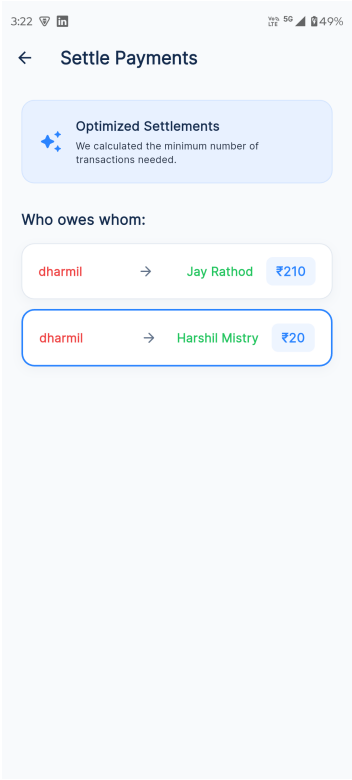


Figure 5: Graph-Optimised Settlement Output

8.3. Sample Transactions

Table 4: Example Group Scenario — Before & After Optimisation

LightBlue **From**	**To**	**Amount ()**	**Description**
Raw Transactions Recorded			
Aman	Harshita	500	Dinner at Café Deluxe
Harshita	Vaibhavi	300	Metro tickets
Vaibhavi	Aman	200	Grocery run
After Algorithm Optimisation (1 transaction instead of 3)			
Aman	Vaibhavi	300	Net settlement

9. Business Model

FinBuddy can generate revenue and sustain itself through a layered monetisation strategy:

1. **Freemium Subscription** — Core group splitting and settlement features remain free. A *FinBuddy Pro* tier (49/month) unlocks unlimited group history, advanced analytics, and priority AI processing.
2. **Transaction Fee on UPI Facilitation** — A micro-fee (0.2–0.5%) on in-app facilitated settlements above a threshold, applicable once transaction volume grows.
3. **Affiliate Partnerships** — Revenue sharing with UPI-enabled wallets (PhonePe, GPay) or travel booking platforms for referral traffic generated by trip-expense groups.

10. Challenges Faced

1. **Settlement Algorithm Correctness** — Ensuring the greedy matching always converges to a valid zero-balance state required careful handling of floating-point rounding; solved by maintaining balances as integer paise (1 rupee = 100 paise).
2. **VLM Receipt Extraction Reliability** — llama-3.2-vl occasionally misread Indian-language receipts or poor-quality images; addressed with structured prompt engineering specifying output schema and fallback manual confirmation.
3. **Real-Time Sync Conflicts** — Multiple users logging expenses simultaneously caused Firestore write conflicts; resolved with atomic Firestore transactions and optimistic UI updates with rollback.
4. **UPI Deep-Link Compatibility** — Android intent parameters and iOS Universal Link behaviour differ across UPI apps (BHIM, PhonePe, GPay); required device-specific deep-link parameter tuning.
5. **Secure API Key Management** — Embedding Deepgram, Gemini, and NIM keys safely without exposure; solved by routing all AI calls through the Express backend and storing keys in server-side `.env`.
6. **Voice Parsing Ambiguity** — Users phrase expenses inconsistently; Gemini prompts were iteratively refined with few-shot examples to handle varied natural-language patterns robustly.

11. Future Scope

1. **Automated UPI Payment Confirmation** — Integrate UPI payment status webhooks so FinBuddy automatically marks settlements complete without manual user confirmation.
2. **Recurring Expense Templates** — Allow groups to define monthly recurring bills (rent, subscriptions) that auto-log on a schedule.
3. **Multi-Currency Support** — Handle international trips with live exchange rates, enabling cross-currency settlements.
4. **Expense Budget Alerts** — Per-category and per-group budget caps with push notification alerts when thresholds are approached.

5. **Bank Statement Import** — Parse PDF or CSV bank statements to auto-log historical group expenses retroactively.
6. **Web Dashboard** — A companion web application for larger event organisers who prefer a desktop interface.
7. **On-Device LLM** — Evaluate running a smaller quantised model on-device for offline voice parsing, reducing dependency on cloud APIs.
8. **ONDC / Account Aggregator Integration** — Leverage India's Open Network for Digital Commerce and AA framework for richer financial data access with user consent.

12. Conclusion

FinBuddy addresses a real and widespread pain point: the friction, confusion, and inequity that arise when groups of people share expenses without a dedicated tracking system. By combining a mathematically sound debt-simplification algorithm, AI-powered multi-modal input methods (voice and VLM-based receipt scanning), and seamless UPI payment integration, FinBuddy delivers an end-to-end solution that is smarter, faster, and more transparent than any existing informal approach.

Built with Flutter for true cross-platform reach, Firebase for real-time data and authentication, and a Node.js backend for secure AI orchestration, the architecture is production-ready and designed for scale. The replacement of classical OCR with a Vision-Language Model (llama-3.2-vl) marks a deliberate design choice to leverage frontier AI capabilities for robustness and accuracy that rule-based pipelines cannot match.

FinBuddy has the potential to become the standard group-finance tool for India's digitally native youth, with a clear path to monetisation and a rich roadmap of features that deepen its utility over time.

13. References

[1] Flutter Documentation. *Flutter — Build apps for any screen.* https://flutter.dev/docs

[2] Firebase. *Firebase Documentation — Firestore & Authentication.* https://firebase.google.com/docs

[3] Deepgram. *Deepgram Speech Recognition API Documentation.* https://developers.deepgram.com

[4] Google. *Gemini API Developer Guide.* https://ai.google.dev/docs

[5] Meta. *llama-3.2-vl Vision Capabilities.* https://www.llama.com/docs/overview/

[6] National Payments Corporation of India. *UPI Developer Documentation.* https://www.npci.org.in/what-we-do/upi/product-overview

[7] Cormen, T.H., et al. *Introduction to Algorithms*, 4th ed. MIT Press, 2022.

[8] Provider Package. *provider 6.1.5 — pub.dev.* https://pub.dev/packages/provider

[9] Dart. *Dart Programming Language Documentation.* https://dart.dev/guides